

Git & GitHub — Complete Full Stack Developer Roadmap

For a **Full Stack Developer**, you don't need to learn every internal detail of Git. You should become strong in **Git fundamentals** + **collaborative GitHub workflows** + **professional project management**.

Here is a complete **Git & GitHub syllabus**, arranged from beginner → advanced → professional.

LEVEL 1 — Git Fundamentals

1. Introduction to Version Control

- What is Version Control?
- Why Version Control is needed
- Problems without Version Control
- Version Control System (VCS)
- Centralized vs Distributed VCS
- Git vs GitHub
- Git vs GitLab vs Bitbucket
- Local repository vs Remote repository

2. Git Installation & Setup

- Installing Git
- Checking Git version
- Git configuration
- git config
- Username configuration
- Email configuration
- Default branch configuration
- Git aliases
- Global vs local configuration
- .gitconfig

3. Understanding Git Internals

Understand this model:



- Working Directory
- Staging Area
- Local Repository

- Remote Repository
- Commit
- HEAD
- Branch
- Hash / SHA
- Git objects — basic understanding

LEVEL 2 — Basic Git Commands

4. Creating a Repository

- git init
- .git directory
- git status

5. Tracking Files

- Untracked files
- Tracked files
- Modified files
- Staged files
- git add
- git add .
- git add -A

6. Commits

- git commit
- git commit -m
- Good commit messages
- Atomic commits
- Commit history
- git log
- git log --oneline
- git show

7. Viewing Changes

- git diff
- git diff --staged
- Comparing commits
- Comparing branches

8. Removing / Renaming Files

- git rm
- git mv
- Removing files from tracking
- File deletion vs Git deletion

LEVEL 3 — Git Ignore & Repository Management

9. .gitignore

Extremely important for full-stack development.

- What is .gitignore?
- Why use it?
- Syntax
- Files vs directories
- Wildcards
- Negation !
- Global .gitignore

Examples of things you normally shouldn't commit:

```
node_modules/  
.env  
.env.local  
dist/  
build/  
*.log  
.DS_Store
```

10. .gitkeep

- Why it exists
- Empty directories
- When to use it

11. .gitattributes

- Line endings
- Binary files
- Git LFS basics

LEVEL 4 — GitHub Fundamentals

12. GitHub Basics

- What is GitHub?
- GitHub repositories
- Public vs Private repositories
- Repository visibility
- README
- Repository description
- Topics
- License
- Releases
- Stars
- Forks
- Watch

13. Connecting Local Git to GitHub

- Create GitHub repository
- Add remote

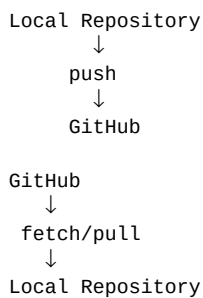
```
git remote add origin <URL>
```

- git remote
- git remote -v
- Change remote URL
- Remove remote

14. Push & Pull

- git push
- git pull
- git fetch
- git clone

Understand:



LEVEL 5 — Branching

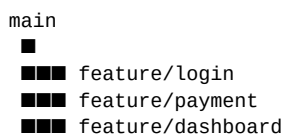
This is one of the **most important Git topics** for professional development.

15. Branch Fundamentals

- What is a branch?
- Why branches?
- git branch
- git switch
- git checkout
- Creating branches
- Deleting branches
- Renaming branches
- Current branch
- HEAD

16. Branching Strategies

Understand:



Branch naming:

```

feature/login
feature/payment
bugfix/navbar
hotfix/security
refactor/auth
  
```

17. Merging

- git merge

- Fast-forward merge
- Three-way merge
- Merge conflicts
- Resolving conflicts
- Abort merge

LEVEL 6 — Merge Conflicts

18. Understanding Conflicts

- Why conflicts occur
- Conflict markers
- Choosing changes
- Manual conflict resolution
- VS Code conflict resolver

Example:

```
<<<<<<< HEAD
your code
=====
their code
>>>>>>> feature-branch
```

19. Conflict Resolution Workflow

```
Pull
↓
Conflict
↓
Open conflicting file
↓
Resolve
↓
git add
↓
git commit
```

LEVEL 7 — Undoing Things

This is where Git becomes much more powerful.

20. Undo Working Directory Changes

- git restore
- Restore a file
- Restore multiple files

21. Unstage Files

```
git restore --staged <file>
```

22. Amend Commits

```
git commit --amend
```

Understand when **not** to use it.

23. Reset

- git reset
- Soft reset
- Mixed reset
- Hard reset

Understand:

```
--soft  
--mixed  
--hard
```

24. Revert

```
git revert
```

Difference:

```
reset → moves history  
revert → creates a new commit that reverses history
```

25. Reflog

```
git reflog
```

- Recover lost commits
- Recover deleted branches

LEVEL 8 — GitHub Collaboration

26. Forking

- What is a fork?
- Fork vs clone
- When to fork
- Syncing a fork

27. Pull Requests

Understand complete PR workflow:

```
Clone  
↓  
Create Branch  
↓  
Make Changes  
↓  
Commit  
↓  
Push  
↓  
Pull Request  
↓  
Code Review  
↓  
Changes Requested  
↓  
Update Branch  
↓  
Approve  
↓  
Merge
```

- Creating PR
- Draft PR
- PR title
- PR description
- Reviewers
- Labels
- Assignees
- Comments
- Requested changes
- Approving PR

- Merging PR
- Closing PR

28. Code Review

- Review code
- Inline comments
- Suggestions
- Review etiquette
- Security review
- Maintainability
- Code quality

LEVEL 9 — Remote Branch Management

29. Fetching

```
git fetch
```

- Fetch vs pull
- Remote-tracking branches
- origin/main

30. Pull Strategies

- git pull
- Pull + merge
- Pull + rebase

31. Tracking Branches

- Upstream branches
- -u
- Tracking relationships

LEVEL 10 — Rebase

32. Rebase Fundamentals

```
git rebase
```

Understand:

Merge:

```
A---B---C
      \
      D---E
```

Rebase:

```
A---B---C---D'---E'
```

33. Interactive Rebase

```
git rebase -i
```

- pick
- reword
- edit
- squash
- fixup

- drop

34. Merge vs Rebase

Know when to use each.

35. Rebase Conflicts

- Resolve conflict
- `git rebase --continue`
- `git rebase --abort`
- `git rebase --skip`

LEVEL 11 — Advanced Git

36. Cherry-Pick

`git cherry-pick`

- Move specific commits
- Hotfixes
- Backporting changes

37. Stash

```
git stash
git stash pop
git stash apply
git stash list
git stash drop
```

Use case:

```
Working on feature A
↓
Urgent bug appears
↓
stash
↓
fix bug
↓
return to feature A
```

38. Tags

`git tag`

- Lightweight tags
- Annotated tags
- Version tagging

Example:

```
v1.0.0
v1.1.0
v2.0.0
```

39. Releases

- Creating releases
- Release notes
- Versioning
- Semantic Versioning

LEVEL 12 — GitHub Professional Features

40. README.md

- Project description
- Features
- Tech stack
- Installation
- Usage
- Screenshots
- API documentation
- Environment variables
- Deployment
- Contributors
- License

41. GitHub Issues

- Create issue
- Labels
- Milestones
- Assignees
- Issue templates
- Bug reports
- Feature requests

42. GitHub Projects

- Kanban boards
- Tasks
- Issues
- Roadmaps
- Sprint planning

43. Discussions

- Q&A;
- Ideas
- Community discussions

44. Wiki

- Documentation
- Technical guides

LEVEL 13 — GitHub Actions / CI-CD

Very important for a modern full-stack developer.

45. CI/CD Fundamentals

```

Developer
↓
git push
↓
GitHub
↓
GitHub Actions
↓
Build

```

↓
Test
↓
Deploy

46. GitHub Actions

- Workflows
- Events
- Jobs
- Steps
- Actions
- Runners
- YAML
- Secrets
- Environment variables

Example:

```
.github/  
  workflows/  
    ci.yml
```

47. CI Pipeline

- Push
- Install dependencies
- Lint
- Test
- Build

48. Deployment Automation

- Vercel
- Netlify
- AWS
- Azure
- GCP
- Render

LEVEL 14 — GitHub Security

49. Secrets

Extremely important.

Never commit:

```
.env  
API keys  
Database passwords  
JWT secrets  
Cloud credentials  
Private keys
```

- GitHub Secrets
- Environment secrets
- Repository secrets
- Organization secrets

50. Accidentally Committed Secrets

- Removing secret from latest commit
- Removing it from history
- Rotating/revoking exposed credentials
- Why simply deleting the file isn't enough

51. Dependabot

- Dependency updates
- Security alerts
- Vulnerability fixes

52. Secret Scanning

- Secret detection
- Push protection
- Credential exposure

LEVEL 15 — Git Internals

You don't need to become a Git developer, but a full-stack developer should understand the fundamentals.

53. Git Objects

- Blob
- Tree
- Commit
- Tag

54. HEAD

Understand:

```
HEAD
↓
Branch
↓
Commit
```

55. SHA

- Commit hashes
- Object identification

56. Git References

- Branch references
- Tags
- HEAD
- Remote references

57. Git Database

Basic understanding of:

```
.git/
■■■ objects/
■■■ refs/
■■■ HEAD
■■■ config
■■■ index
■■■ logs/
```

LEVEL 16 — Professional Git Workflow

58. Git Flow

Understand:

```
main
develop
feature
release
hotfix
```

59. GitHub Flow

Simpler workflow:

```
main
↓
feature branch
↓
Pull Request
↓
Review
↓
Merge
```

60. Trunk-Based Development

- Short-lived branches
- Frequent integration
- Feature flags
- Continuous integration

61. Commit Conventions

Learn:

```
feat: add login system
fix: resolve navbar bug
docs: update README
refactor: simplify authentication
test: add login tests
chore: update dependencies
```

This is **Conventional Commits**.

LEVEL 17 — Git for Full-Stack Projects

Frontend

```
React
Next.js
HTML/CSS/JS
Tailwind
```

Backend

```
Node.js
Express
Java
Spring Boot
REST APIs
```

Database

```
MongoDB
PostgreSQL
MySQL
```

DevOps

```
Docker
CI/CD
Cloud deployment
```

LEVEL 18 — Monorepos

Understand the concept of:

```
project/  
  frontend/  
  backend/  
  shared/  
  README.md
```

- Monorepo
- Polyrepo
- Git workspaces — basic awareness
- Shared code
- Repository structure

LEVEL 19 — Git LFS

- What is Git LFS?
- Why large files are problematic
- Tracking large files
- Git LFS workflow

Useful for:

- Videos
- Large datasets
- ML models
- Large binary files

LEVEL 20 — Troubleshooting

You should be able to solve:

- "I committed the wrong file."
- "I accidentally deleted my branch."
- "I pushed a secret/API key."
- "My branch has conflicts."
- "I need to undo my last commit."
- "I need one old commit from another branch."
- "I need to temporarily save my current work."
- "My local branch is behind GitHub."
- "I committed to the wrong branch."
- "I need to recover a lost commit."

LEVEL 21 — Must-Know Command Set

As a full-stack developer, these should become second nature:

```
git init  
git clone  
  
git status  
  
git add  
git commit  
  
git log
```

```
git show
git diff

git branch
git switch
git checkout

git merge
git rebase

git remote
git fetch
git pull
git push

git stash

git reset
git restore
git revert

git reflog

git cherry-pick

git tag
```

LEVEL 22 — GitHub Skills You Should Master

You should be comfortable with:

```
GitHub Repository
■
■■■ README
■■■ Issues
■■■ Projects
■■■ Pull Requests
■■■ Actions
■■■ Releases
■■■ Discussions
■■■ Security
■■■ Settings
```

And especially:

Repository → Branch → Commit → Push → Pull Request → Review → Merge → CI/CD → Release

What You Actually Need for Placements

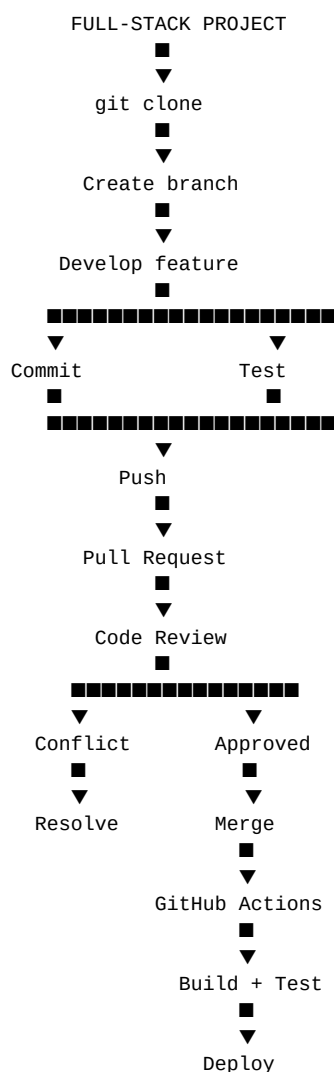
Don't spend months learning Git internals. For a student/full-stack developer, prioritize like this:

Priority	Topics
■ Must Master	init, clone, status, add, commit, push, pull
■ Must Master	branch, switch, merge
■ Must Master	GitHub repositories
■ Must Master	Pull Requests
■ Must Master	Merge conflicts
■ Must Master	.gitignore
■ Must Master	fetch vs pull
■ Must Master	reset vs revert vs restore
■ Strong	rebase
■ Strong	stash
■ Strong	cherry-pick
■ Strong	reflog

Priority	Topics
■ Strong	GitHub Issues/Projects
■ Strong	GitHub Actions
■ Strong	GitHub security/secrets
■ Basic	Git internals
■ Basic	Git LFS
■ Basic	Git Flow
■ Basic	Monorepos

Your Full-Stack Git Skill Target

By the end, you should be able to independently do this:



For your current Full Stack + Java backend path, I would put the highest priority on **Git fundamentals** → **branching** → **merge/conflicts** → **GitHub** → **PR/code review** → **rebase** → **GitHub Actions** → **security**. You can leave deep Git internals for later.